

Buffered Byte Streams

Optimizing Binary I/O Performance

codehive

codehive

1. The Slow Approach (Unbuffered)

Reading and writing one byte at a time is simple but inefficient for large files because it triggers a disk access for every single byte.

```
int b;
while ((b = input.read()) != -1) {
    output.write(b);
}
```

2. The Fast Approach (Buffered)

By wrapping `FileInputStream` inside a `BufferedInputStream`, Java reads large blocks of data (usually 8KB) into memory at once.

```
try (BufferedInputStream bis = new BufferedInputStream(new
FileInputStream("image.jpg"));
    BufferedOutputStream bos = new BufferedOutputStream(new
FileOutputStream("copy.jpg"))) {

    byte[] buffer = new byte[1024];
    int bytesRead;
    while ((bytesRead = bis.read(buffer)) != -1) {
        bos.write(buffer, 0, bytesRead);
    }
}
```

Why use Buffering?

- **Reduced Overhead:** Fewer calls to the OS kernel.
- **Block Transfer:** Hard drives and SSDs are optimized to read blocks, not single bytes.
- **Stability:** Less CPU usage during heavy file transfers.

Efficiency Engineering with

codehive

codehive